

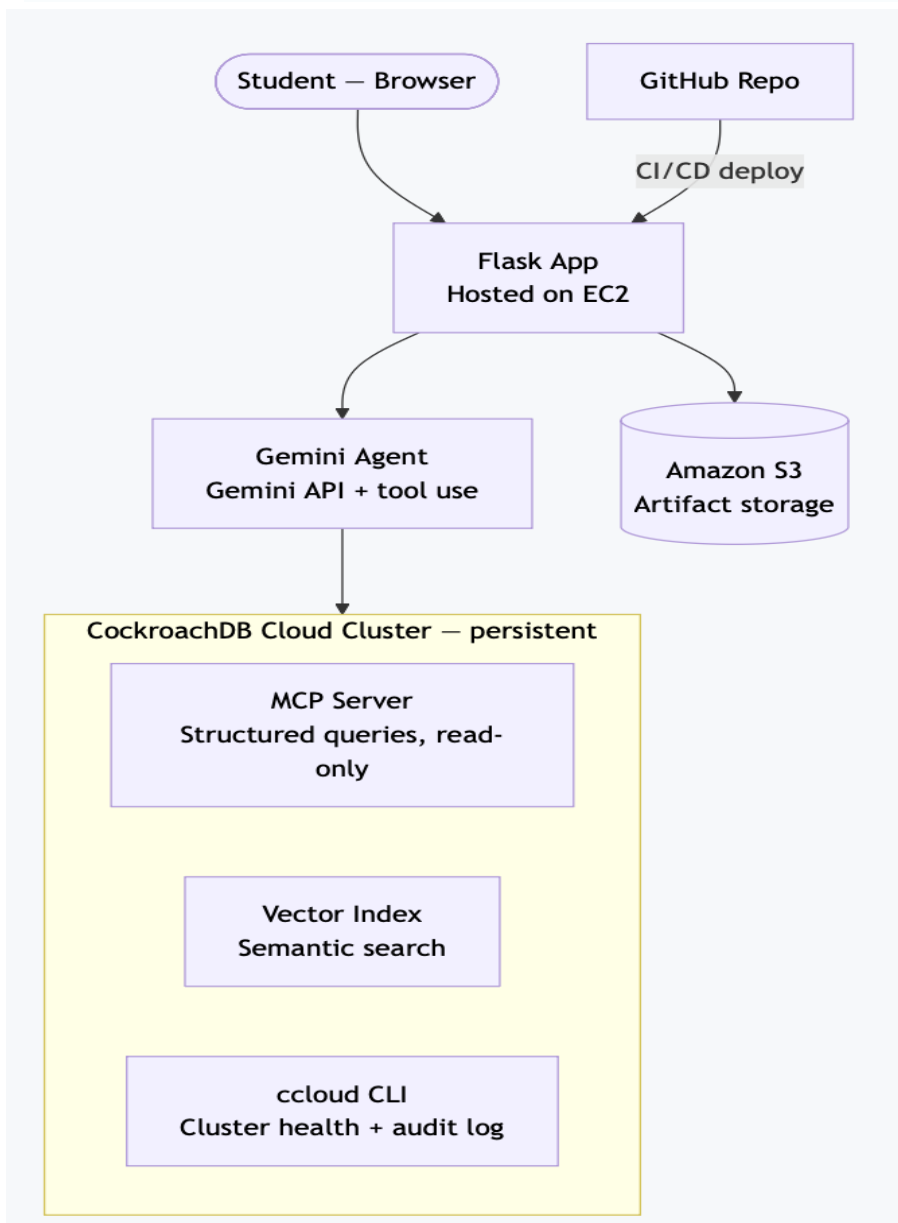
AI Course Advisor — Architecture

An agentic course-recommendation platform that uses CockroachDB as its persistent memory layer to help students choose courses based on their interests, target career, or GPA goals.

1. Overview

A student describes what they want (an interest, a target job, a GPA to hit). A Gemini agent reasons over structured university data and semantic course/career matches — both stored in a single CockroachDB Cloud cluster — and returns a ranked, explained set of course recommendations. The cluster is the system's memory: it holds conversation history, student context, in-progress task state, course/career embeddings, and all transactional enrollment data.

2. Architecture Diagram



3. Components

Component	Role
Student (browser)	Submits a goal — interest, target job, or GPA target — through a simple form.
Flask app (Amazon EC2)	Request handling, session management, calls the Gemini agent, writes artifacts to S3.
Gemini agent (Gemini API)	Orchestrates tool calls, reasons over structured + semantic results, produces the final recommendation with a short "why this helps you" explanation per course.
CockroachDB Cloud cluster	The persistent memory layer — one cluster, three access paths (MCP, vector index, ccloud CLI).
Amazon S3	Stores generated recommendation reports and dataset backups — the AWS service requirement.
GitHub → AWS EC2	CI/CD: Deployed via github actions

4. Memory layer mapping

This is the part judges will look at closest — memory isn't one thing here; it's five, all living in CockroachDB.

Memory type	Where it lives	How the agent uses it
Conversation history	conversations table (session_id , turn , role , content , created_at)	Agent reads prior turns via MCP so a returning student gets continuity instead of starting cold.
User context	students + student_goals tables (interests, target job, target GPA, completed courses)	Read via MCP at the start of every session to personalize the query.
Task state	recommendation_sessions table (status , last_tool_used , partial_results , updated_at)	Written mid-run so a request survives an EC2 restart, worker timeout, or retry without losing progress — this is the "memory that never goes down" in practice.
Embeddings	VECTOR columns on courses and career_outcomes	Queried through the distributed vector index for semantic interest-to-course and course-to-career matching.
Structured transactional data	enrollments , prerequisites_graph , graduation_requirements , course_success_rates	Queried read-only via MCP for eligibility checks, prerequisite validation, and GPA-path feasibility.

5. CockroachDB tools — 3 of 4

(Agent Skills Repo was the 4th option, not used — see cost/time tradeoff discussion in the build log.)

Tool	What it does here
MCP Server	Live, read-only structured queries against enrollment, prerequisite, and requirement tables. Full audit logging, no custom proxy.
Distributed Vector Indexing	Semantic matching between a student's free-text goal and course/career embeddings, without a separate vector store.
ccloud CLI	Flask shells out to <code>ccloud cluster info <name> -o json</code> and <code>ccloud audit list --limit 5 -o json</code> ; the agent reports cluster health/audit status on request, demonstrating the memory layer is monitored, not just used.

6. AWS services

- **Amazon S3:** stores generated recommendation reports and CSV dataset backups via `boto3`
- **Amazon EC2:** Hosts the Flask backend web application and CI/CD runner.

7. Request lifecycle

1. Student submits a goal $\rightarrow$ Flask writes a new row to `recommendation_sessions` (task-state write #1) and appends to `conversations`.
2. Flask calls the Gemini agent with three tools registered: MCP query, vector search, ccloud health check.
3. Agent calls MCP to check eligibility/prerequisites (memory read).
4. Agent calls the vector tool to semantically match the goal to courses/careers (memory read).
5. Optionally, agent calls ccloud CLI to confirm cluster health before returning results.
6. Agent returns ranked recommendations with short explanations.
7. Flask updates `recommendation_sessions` to `complete` (task-state write #2), appends the agent's reply to `conversations`, and writes a copy of the report to S3.
8. Flask renders results to the student.

8. Deployment

- **Code:** GitHub, public repo, MIT or Apache 2.0 license visible in the About section.
- **Hosting:** Amazon EC2 (`t2.micro` / `t3.micro` , 750 free tier hours/month). Auto-deploys on push to `main` (via GitHub Actions / CI/CD).
- **AWS Compliance:** Fully satisfies both the *"deployed on AWS"* requirement (hosted on EC2) and the *"use \$|ge\$1 AWS service"* requirement (EC2 & Amazon S3).

9. Tech stack

Layer	Choice
Agent	Gemini API (tool use)
Backend	Flask (Python)
Database	CockroachDB Cloud (Basic/serverless)
Embeddings	sentence-transformers, <code>all-MiniLM-L6-v2</code> , run locally (free)
Object storage	Amazon S3
Hosting	Amazon EC2
Source control	GitHub

10. Estimated cost

Item	Cost
CockroachDB Cloud (Basic)	Free — 50M RUs + 10 GiB/month included
Gemini API	Free tier
Amazon S3	Cents — well under free-tier/credit allowances
Amazon EC2	Free tier
Total	Under \$10 for the whole hackathon

